TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

3 créditos

Profesores:

Ing. Jorge Arun Zambrano Cedeño, Mg.

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS: DIANA CAROLINA QUIROGA CHANCAY

PARALELO: A

SEMESTRE: Quinto

ACTIVIDAD:

Actividad 1. Estudio de caso: Sistema de control de rescate y adopción animal

PERÍODO SEPTIEMBRE 2026 - ENERO 2027

Actividades a desarrollar en la Unidad 1

Actividad # 1. Estudio de caso: Sistema de Control de Rescate y Adopción Animal

Diseño lógico y modelado del sistema

1. Análisis del caso

Se propone diseñar un sistema para un refugio que atiende reportes de animales rescatados, mantiene un expediente por caso y registra la asignación de personal responsable. El propósito es evitar que la información quede dispersa y permitir consultar el avance de cada rescate hasta su posible adopción.

El trabajo corresponde a la fase de diseño: se describen las funciones y la estructura que tendría el sistema. No se desarrolla una aplicación ni se implementa una base de datos en esta actividad.

1.1. Alcance

El sistema contempla el registro de reportes, la apertura de casos, la clasificación por especie (perro, gato o fauna silvestre), la asignación de responsables, la evaluación veterinaria, el seguimiento y el cierre mediante adopción cuando corresponda. En este ejercicio, el flujo de adopción se aplica a animales domésticos; la fauna silvestre queda registrada para seguimiento y derivación a atención autorizada, sin modelar su adopción.

1.2. Actores

Actor	Participación prevista
Ciudadano / reportante	Informa el hallazgo y consulta el estado del reporte.
Voluntario	Da seguimiento al caso asignado y registra novedades.
Veterinario	Evalúa el animal y actualiza la información clínica básica.
Administrador del refugio	Revisa ingresos, asigna responsables, habilita procesos de adopción y registra cierres.

1.3. Requerimientos funcionales

Código	Funcionalidad
RF-01	Registrar un reporte de rescate con fecha, ubicación, especie y descripción.
RF-02	Crear el caso y su expediente según el tipo de animal.
RF-03	Permitir al administrador asignar voluntario o veterinario.
RF-04	Registrar seguimiento del rescate y evaluación veterinaria.
RF-05	Actualizar y consultar estados del caso.
RF-06	Notificar a responsables cuando se ingrese o asigne un caso.
RF-07	Marcar como disponible para adopción solo a animales domésticos que cumplan las condiciones definidas.
RF-08	Registrar una adopción y dejar constancia del cierre del caso.

1.4. Requerimientos no funcionales

RNF-01. Mantenibilidad: separar clases y responsabilidades para facilitar cambios sin afectar otros módulos.

RNF-02. Escalabilidad: permitir ampliar el número de registros y usuarios sin rediseñar la estructura básica.

RNF-03. Trazabilidad: conservar el historial de cambios de los casos y las versiones del diseño en Git.

RNF-04. Seguridad: diferenciar las funciones disponibles según el rol y restringir los datos de contacto.

RNF-05. Usabilidad: organizar pantallas y formularios sencillos para disminuir errores de registro.

1.5. Reglas consideradas en el modelo

Como criterio propuesto, un caso comienza en estado Rescatado, puede pasar a En tratamiento, y si es un animal doméstico apto, puede figurar como Disponible para adopción después de la revisión correspondiente. La adopción formalizada cambia el estado a Adoptado. Para fauna silvestre se contempla la derivación, sin adopción. Es un flujo ilustrativo, no un protocolo oficial.

2. Modelado UML

2.1. Diagrama de casos de uso

El diagrama representa las interacciones principales entre las personas y el sistema. El ciudadano reporta y consulta; el administrador coordina los casos; el voluntario y el veterinario registran actividades según sus funciones.

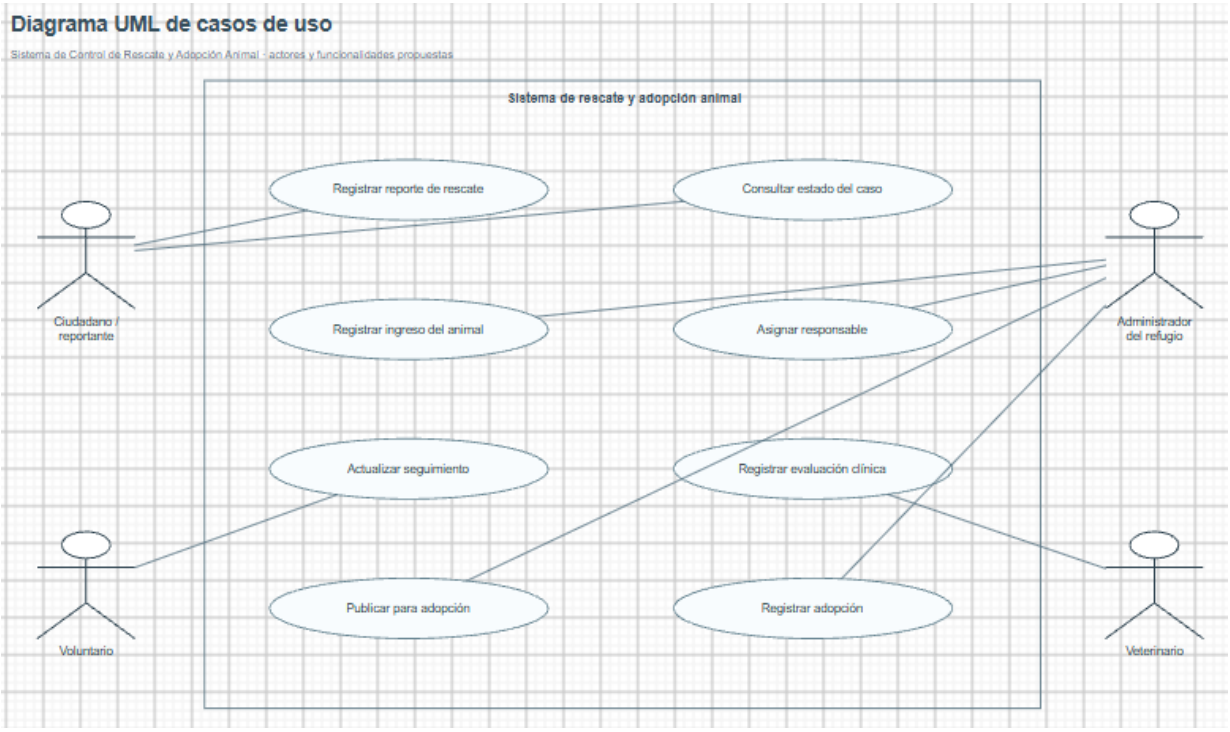


Figura 1. Casos de uso propuestos para el sistema.

2.2. Diagrama de clases: estructura principal

La clase CasoRescate reúne la identificación, la fecha y el estado. Se relaciona con Reportante, Administrador, Expediente y Responsable. La información específica del animal y del personal se mantiene en clases independientes.

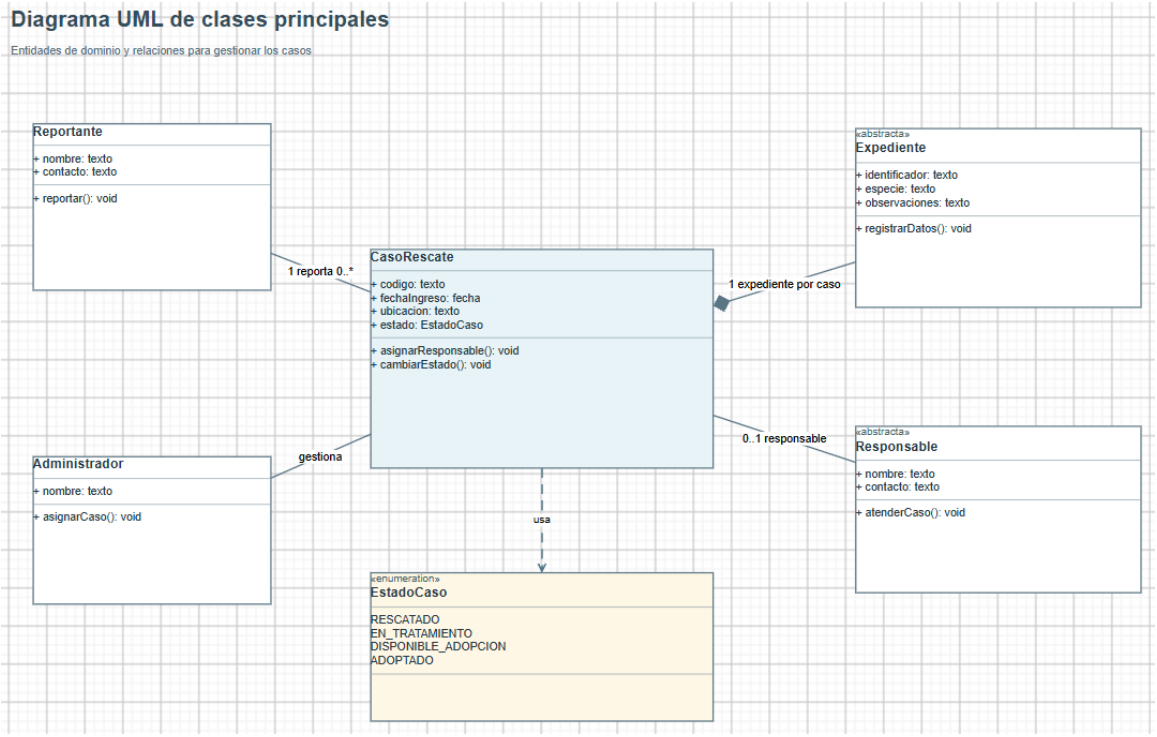


Figura 2. Clases principales y asociaciones UML.

2.3. Especialización de los expedientes y responsables

La clase Expediente permite definir tipos específicos para perros, gatos y fauna silvestre. Responsable se especializa en Voluntario y Veterinario. Esta separación evita asignar a todas las especies o personas atributos que no correspondan.

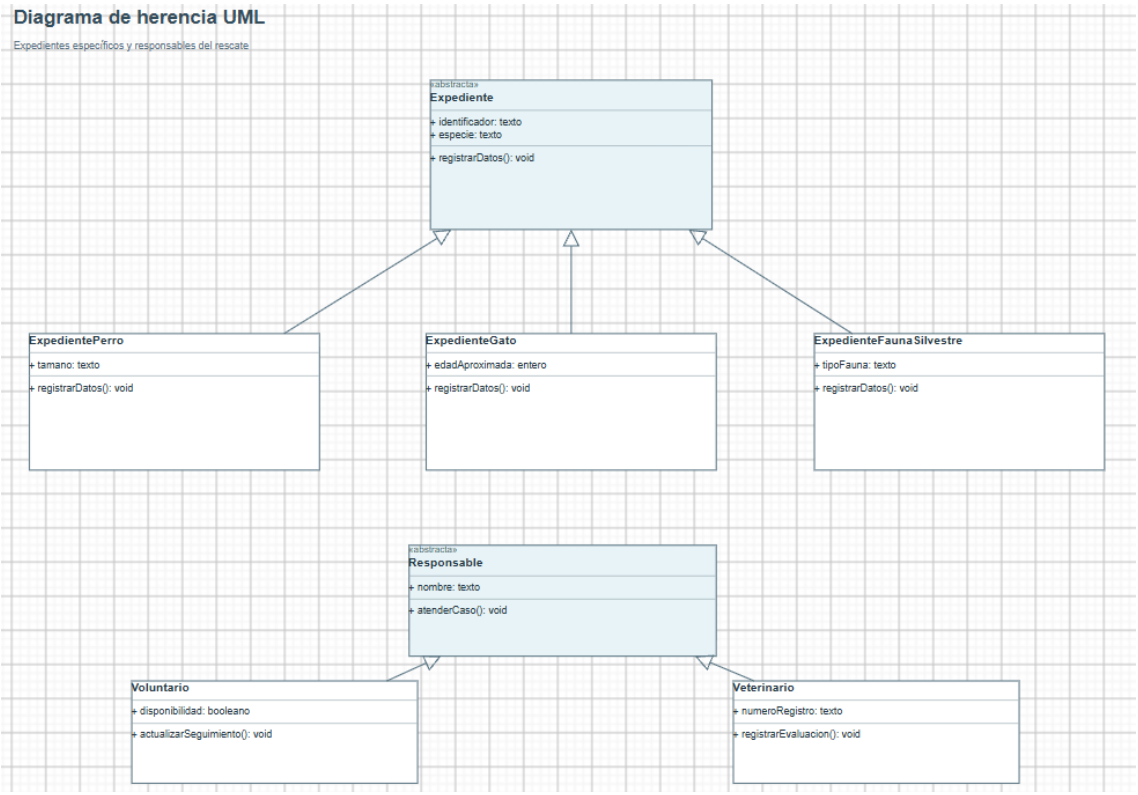


Figura 3. Clases especializadas mediante herencia.

3. Arquitectura lógica propuesta

La arquitectura se organiza en presentación, control, lógica del negocio y persistencia de datos. Las pantallas reciben solicitudes; el controlador valida los datos; la lógica coordina casos, expedientes y avisos; y la persistencia conserva los registros. Esta división facilita que en unidades posteriores se implemente una solución web sin mezclar todas las funciones.

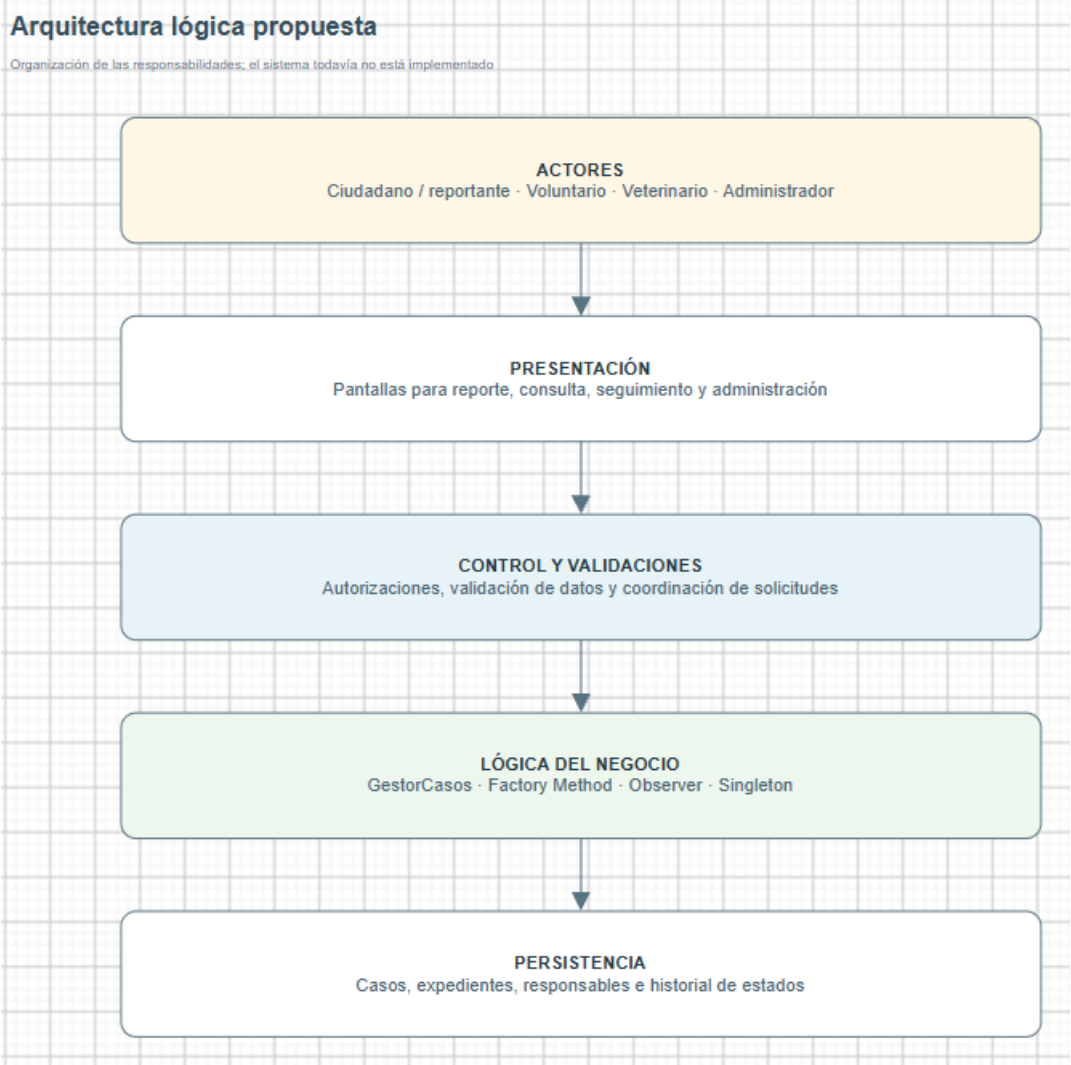


Figura 4. Organización lógica prevista para el sistema.

3.1. Principios de diseño aplicados

Responsabilidad única (SOLID): CasoRescate gestiona el estado; CreadorExpediente determina el tipo de expediente y NotificadorRescates envía avisos. Cada clase tiene un motivo principal de cambio.

Abierto/cerrado (SOLID): al agregar una nueva especie se podría incorporar un creador y su expediente, en lugar de modificar todas las clases ya existentes.

DRY y KISS: el procedimiento de creación y registro se centraliza y el diseño utiliza únicamente las clases necesarias para el alcance definido.

4. Patrones de diseño seleccionados

4.1. Factory Method - patrón creacional

Problema: los expedientes necesitan datos distintos según la especie. Crear todos los tipos directamente dentro de CasoRescate produciría muchas condiciones y dificultaría agregar otros animales.

Solución: una clase abstracta CreadorExpediente declara crear(); cada creador concreto devuelve el expediente adecuado. Así, el caso solicita un expediente sin depender de los detalles de construcción de cada especie.

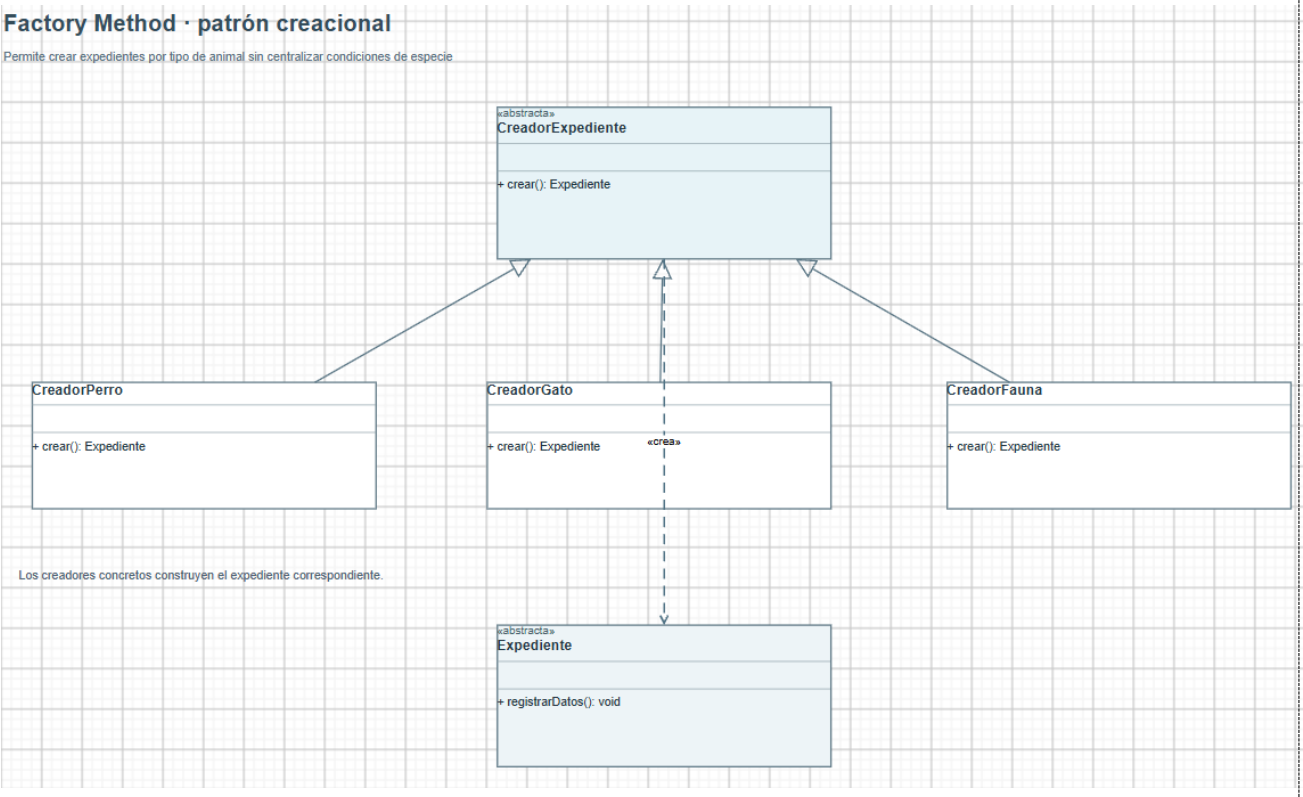


Figura 5. Aplicación propuesta de Factory Method.

4.2. Observer - patrón de comportamiento

Problema: al registrar o asignar un caso, los voluntarios y veterinarios interesados necesitan enterarse sin consultar constantemente la plataforma.

Solución: NotificadorRescates mantiene una lista de observadores. Cuando ocurre un nuevo ingreso o una asignación, invoca actualizar(caso) en los observadores registrados. El sistema puede incorporar otros receptores sin cambiar la operación de registro del rescate.

4.3. Singleton - patrón creacional

Problema: durante el ejercicio se necesita un punto de coordinación para registrar casos y activar notificaciones, sin crear gestores distintos en diferentes secciones de una misma aplicación.

Solución: GestorCasos expone obtenerInstancia(), de modo que en una instancia de la aplicación se comparta un coordinador del proceso. Esto no supone una única instancia global si el sistema se despliega más adelante en varios servidores.

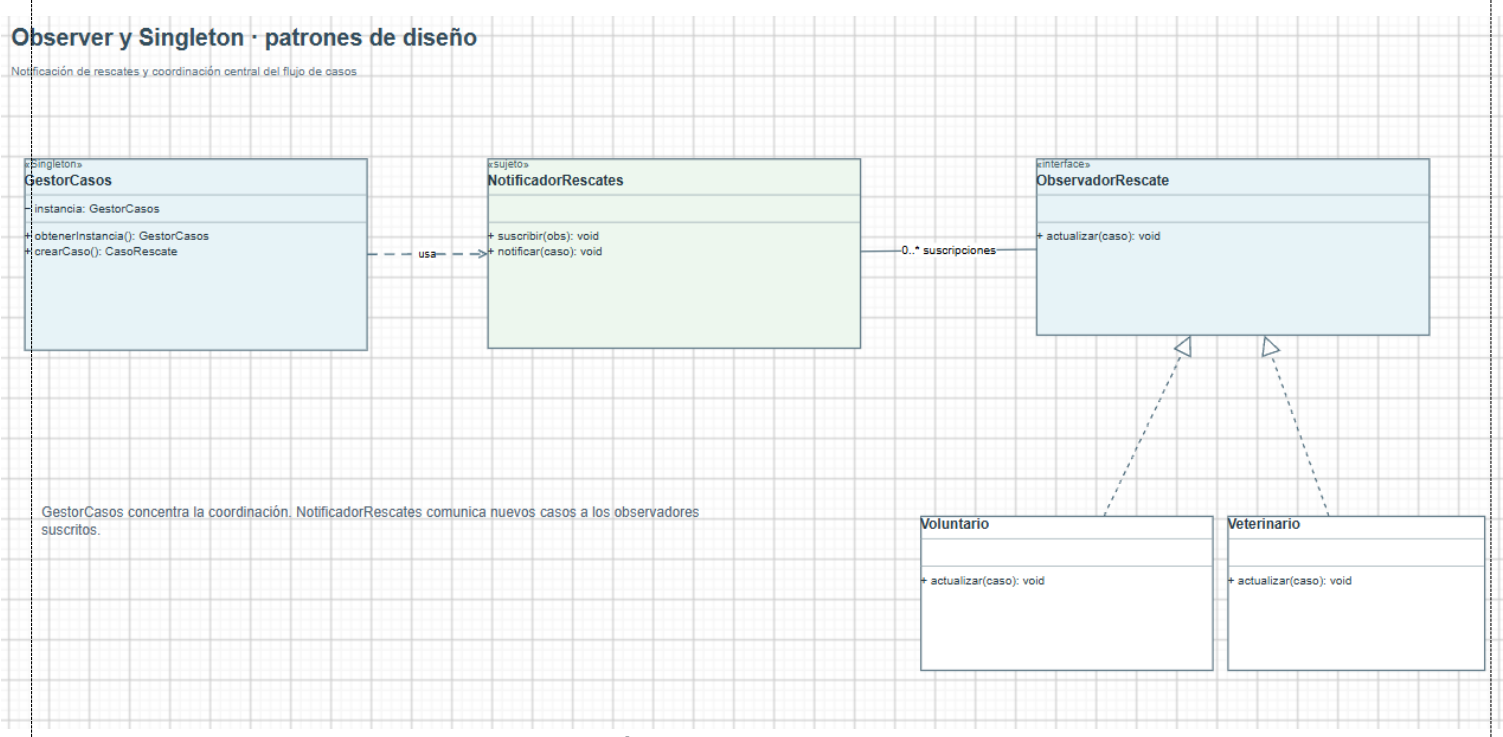
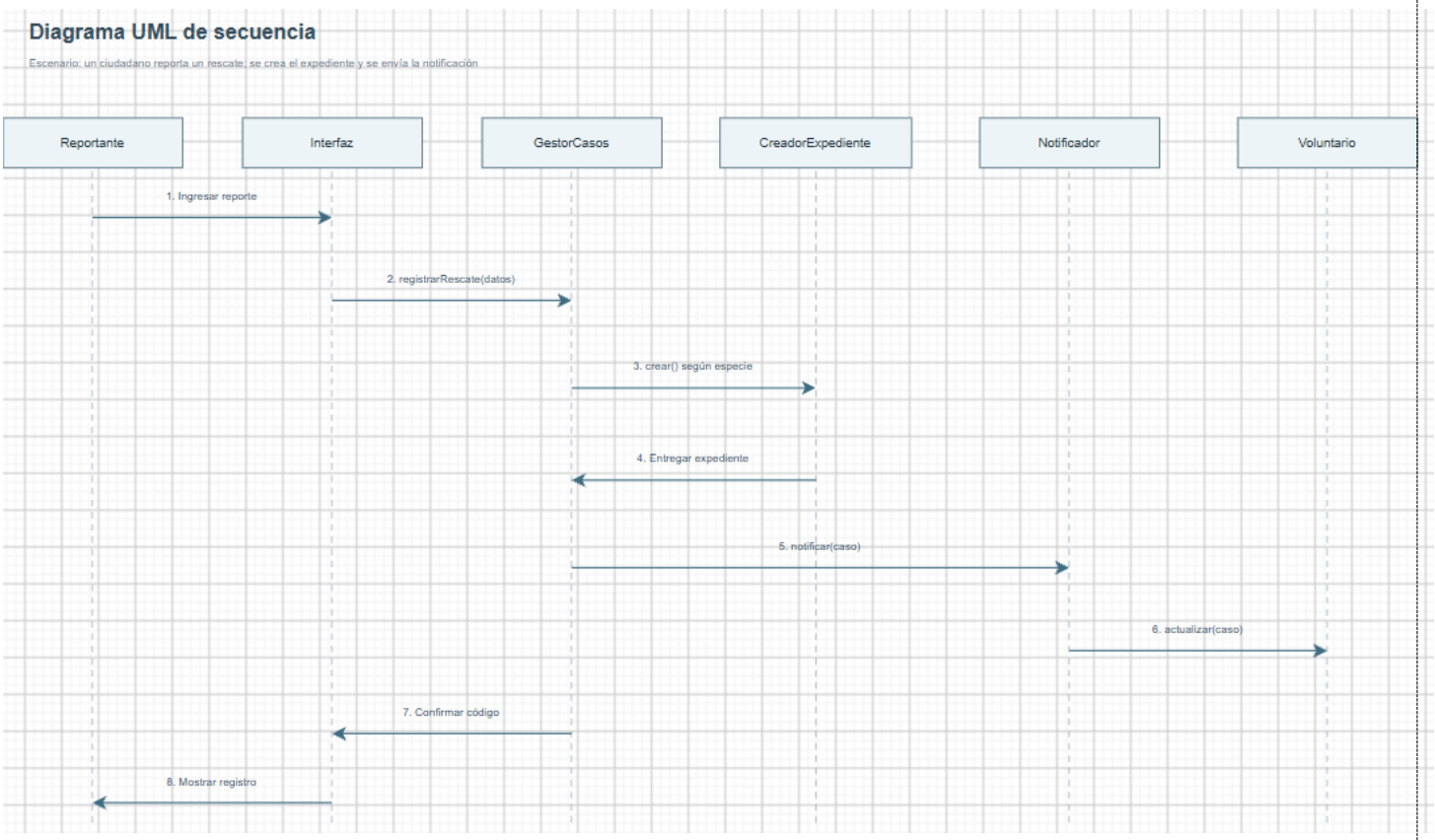


Figura 6. Relación conceptual de Observer y Singleton.

5. Diagrama de secuencia

El flujo comienza con el reporte de un ciudadano. La interfaz remite los datos al gestor de casos, este obtiene el tipo de expediente mediante Factory Method y solicita al



notificador que avise a los observadores registrados. Finalmente se confirma al ciudadano el código del caso.

Figura 7. Secuencia de creación de un nuevo caso y aviso al responsable.

La secuencia representa la aplicación conjunta de los patrones seleccionados. No implica que se hayan ejecutado servicios ni procesos reales; es el comportamiento esperado a partir del diseño.

6. Control de versiones con Gitflow

Git permite mantener un historial de cambios con su autor, fecha y descripción. Se propone manejar main para la versión de entrega, develop para integrar avances y feature/modelado-uml para preparar los diagramas. Antes de entregar se revisa el resultado y se integra a main.

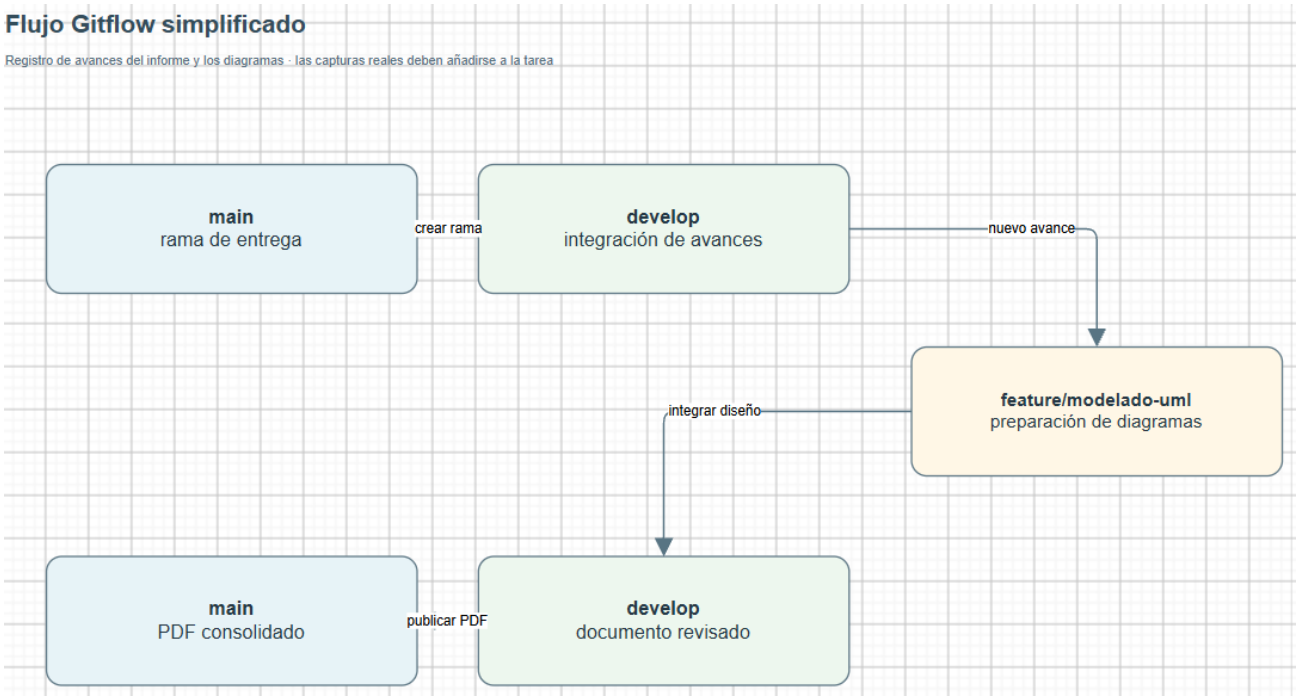


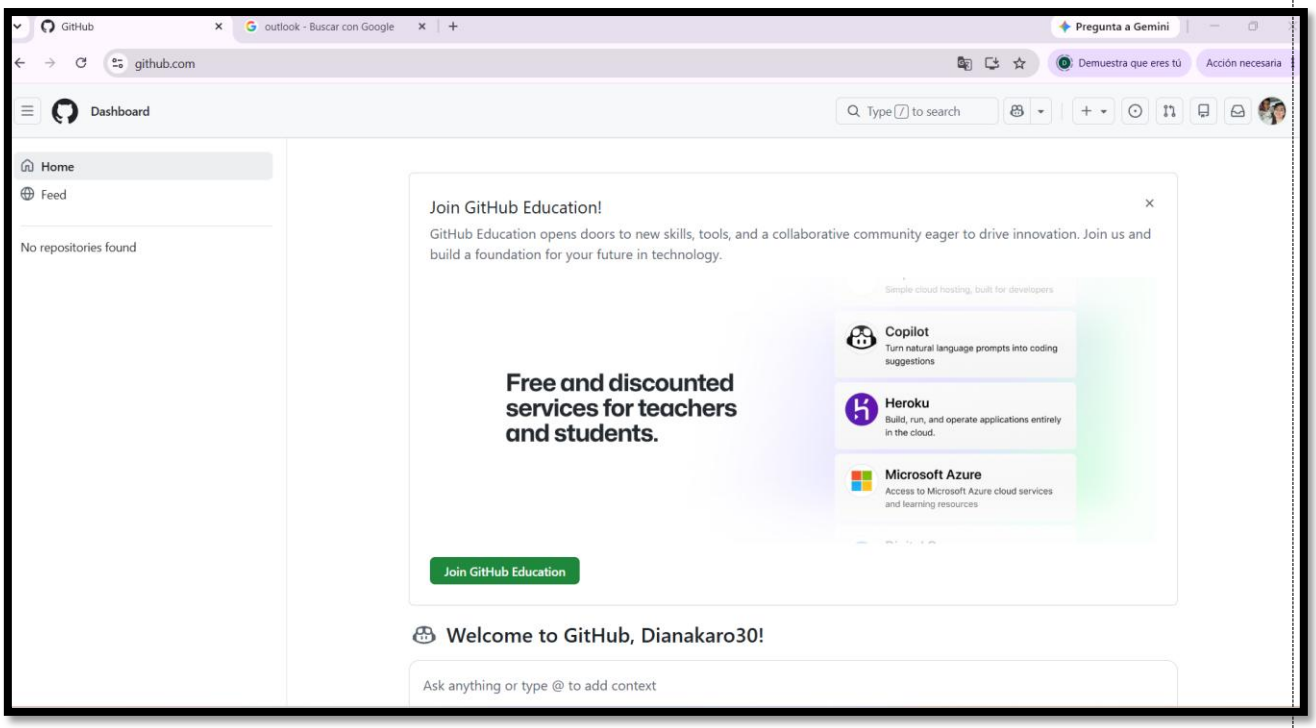
Figura 8. Flujo de trabajo Gitflow simplificado propuesto.

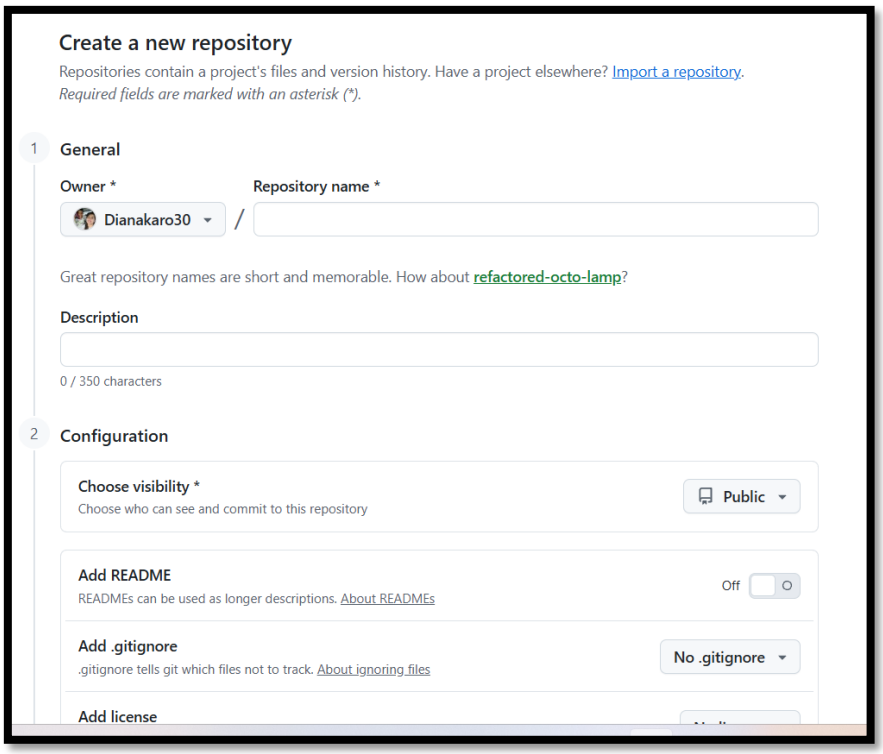
En la práctica, el profesor explicó que para esta primera actividad también acepta la rama principal. Se plantea aquí un Gitflow reducido para conservar coherencia con la guía técnica y dejar una estructura útil para las siguientes actividades.

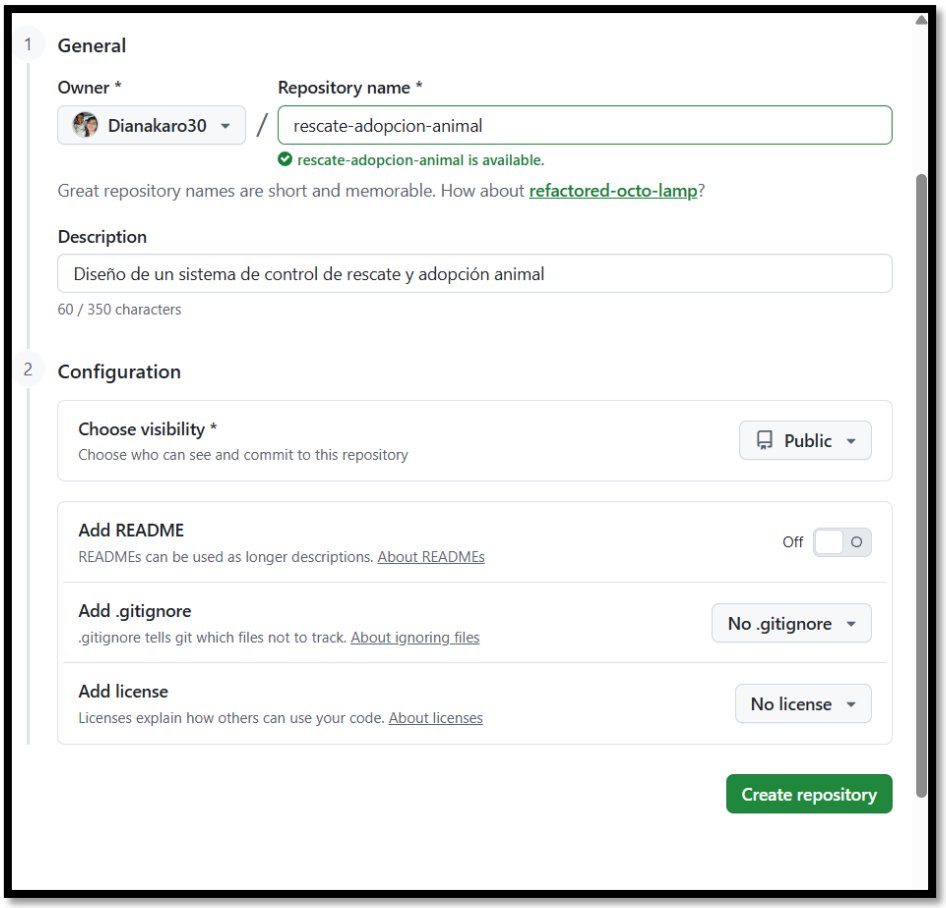
Evidencias

7. Evidencias de la práctica con Git y GitHub

GITHUB







URL del repositorio público:

https://github.com/Dianakaro30/rescate-adopcion-animal.git

Evidencia 1. Inicialización local y consulta del estado de Git (git init y git status).

```
usuario@DESKTOP-BH114M9 UCRT64 ~/Proyectos/rescate-adopcion-animal
$ git init
Initialized empty Git repository in C:/Users/usuario/Proyectos/rescate-adopcion-animal/.git/

usuario@DESKTOP-BH114M9 UCRT64 ~/Proyectos/rescate-adopcion-animal (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        Actividad1_Rescate_Animal_Editable.pdf

nothing added to commit but untracked files present (use "git add" to track)

usuario@DESKTOP-BH114M9 UCRT64 ~/Proyectos/rescate-adopcion-animal (master)
$
```

Evidencia 2. Preparación de los archivos y registro del primer commit.

```
$ git add .

usuario@DESKTOP-BH114M9 UCRT64 ~/Proyectos/rescate-adopcion-animal (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   Actividad1_Rescate_Animal_Editable.pdf


usuario@DESKTOP-BH114M9 UCRT64 ~/Proyectos/rescate-adopcion-animal (master)
$ git commit -m "Primer avance del diseño del sistema de rescate animal"
[master (root-commit) dfb7b91] Primer avance del diseño del sistema de rescate animal
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 Actividad1_Rescate_Animal_Editable.pdf

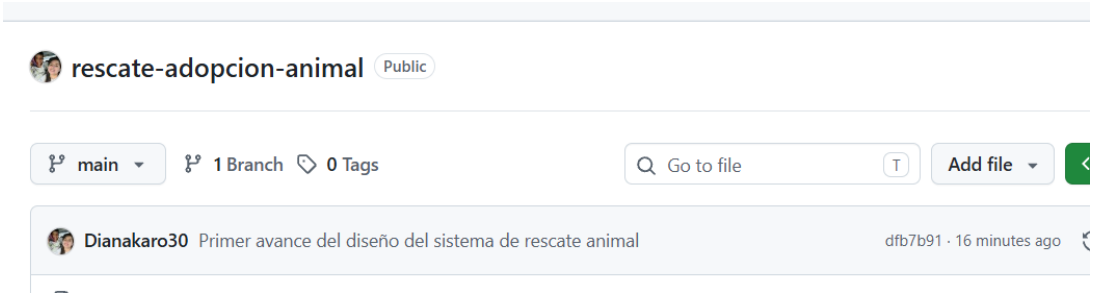
usuario@DESKTOP-BH114M9 UCRT64 ~/Proyectos/rescate-adopcion-animal (master)
$
```

Evidencia 3. Publicación del primer avance en la rama main mediante git push.

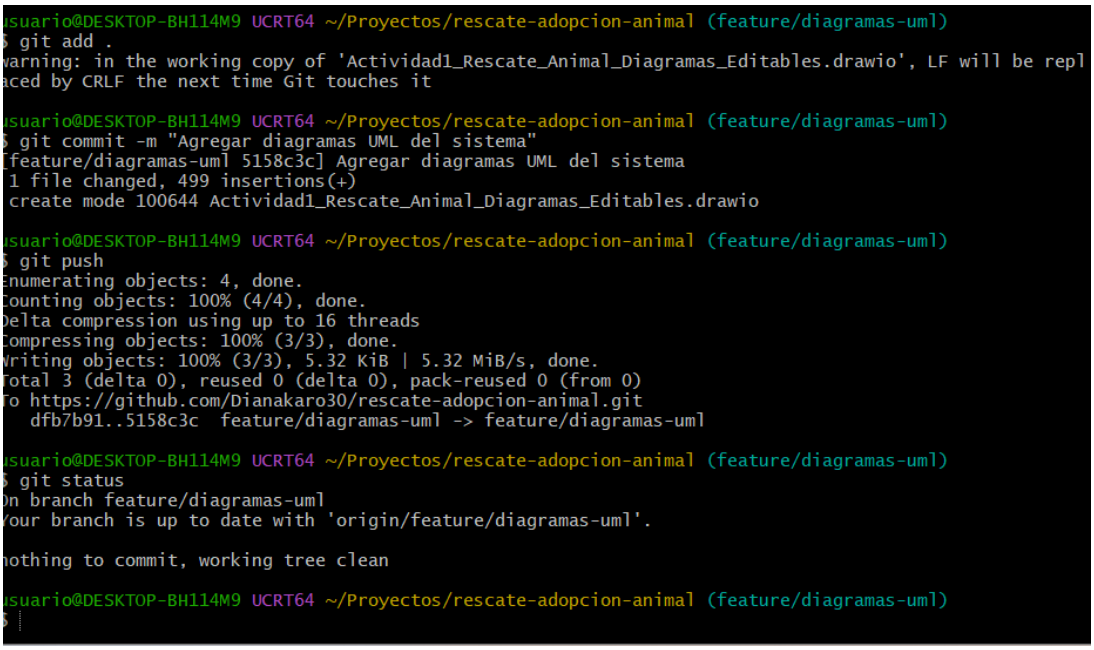
```
usuario@DESKTOP-BH114M9 UCRT64 ~/Proyectos/rescate-adopcion-animal (main)
$ git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 661.33 KiB | 31.49 MiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Dianakaro30/rescate-adopcion-animal.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

usuario@DESKTOP-BH114M9 UCRT64 ~/Proyectos/rescate-adopcion-animal (main)
$
```

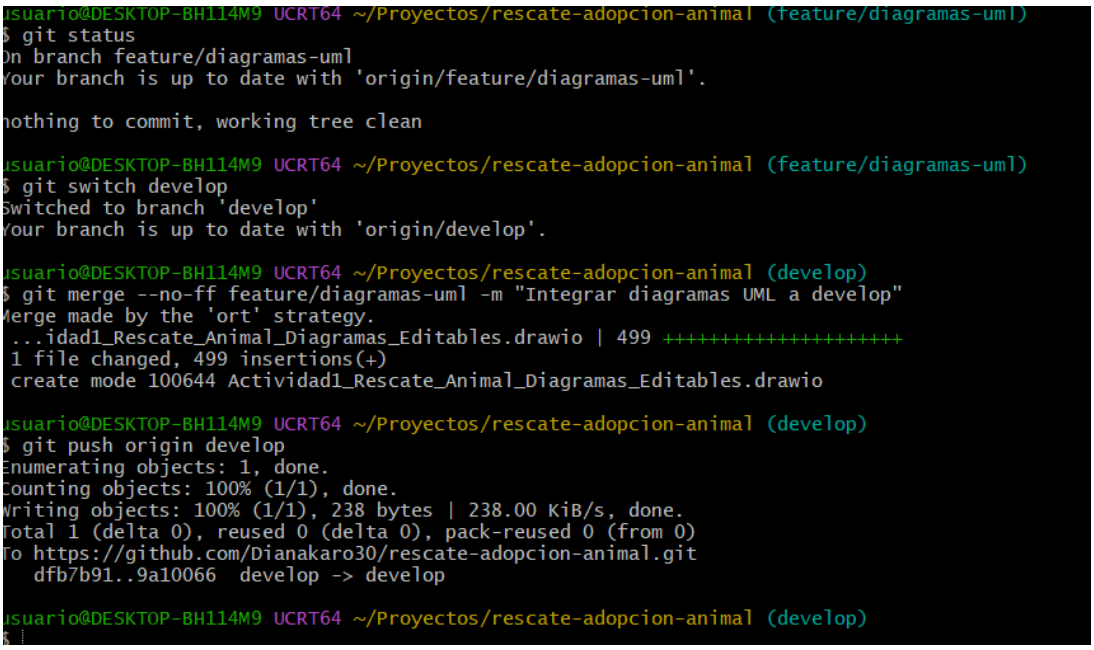
Evidencia 4. Repositorio público en GitHub con el PDF y el primer commit.



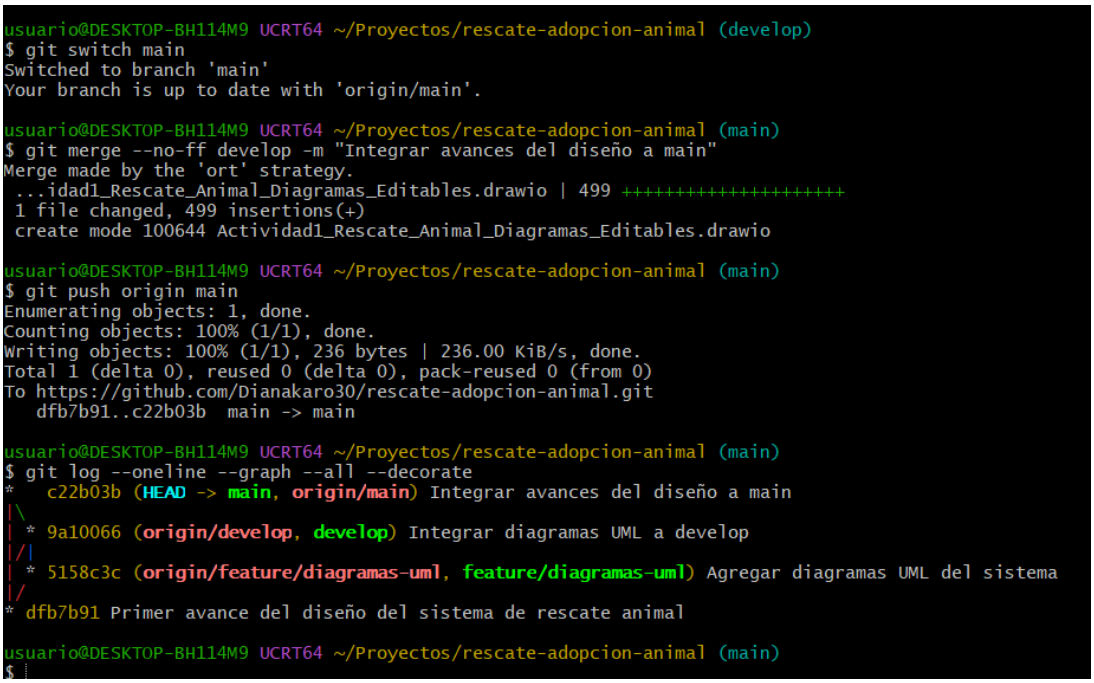
Evidencia 5. Registro y publicación del archivo editable de diagramas en feature/diagramas-uml.



Evidencia 6. Integración de la rama feature/diagramas-uml en develop y subida a GitHub.



Evidencia 7. Integración final en main e historial gráfico de ramas y commits.



El archivo PDF final de la actividad debe actualizarse también en la rama main del repositorio antes de presentar el enlace y el PDF en el aula virtual.

Bibliografía

Universidad Técnica de Manabí. (2026). Guía Actividad DSI Unidad 1: Estudio de caso - Sistema de Control de Rescate y Adopción Animal [Guía de actividad].

Universidad Técnica de Manabí. (2025). Compendio DSI Unidad 1: Desarrollo de software [Material de estudio]

Zambrano Cedeño, J. A. (2026). Clases 1, 2 y 3 de la Unidad 1: Desarrollo de Sistemas Informáticos [Grabaciones de clase]. Universidad Técnica de Manabí.

Zambrano Cedeño, J. A. (2026, 14 de octubre). Tutoría de la Unidad 1 [Grabación]. Universidad Técnica de Manabí.

Universidad Técnica de Manabí. (s. f.). Tutorial de Git y GitHub: Guía paso a paso con solución de problemas frecuentes [Instructivo de clase].